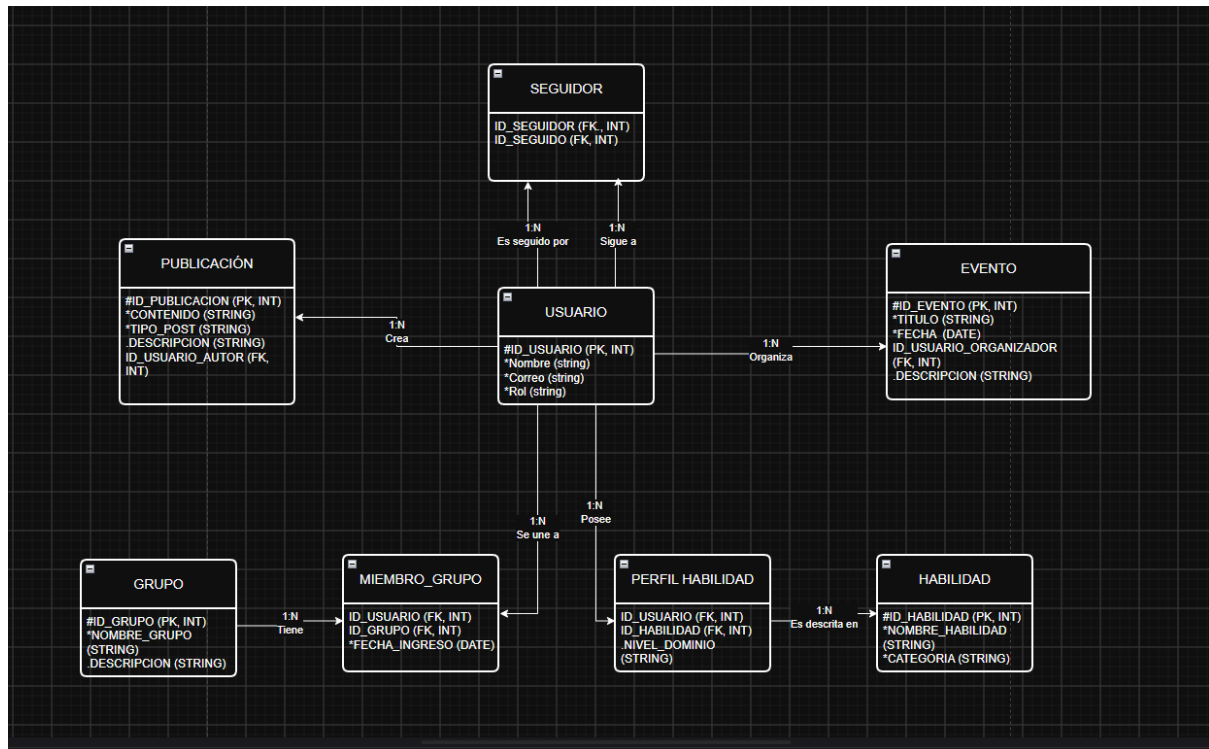


**Informe de Diseño: Modelo entidad relación para la Red Social
Pascualina**

Asignatura: Bases de Datos I

Integrantes: Javer Stiven Calle Ocampo (Trabajo individual)



Introducción

La comunidad estudiantil necesita una plataforma que facilite la comunicación casual, la creación de grupos de estudio, la divulgación de eventos y la conexión para mentorías entre compañeros. Las plataformas institucionales actuales son muy formales y no cubren esta necesidad.

El objetivo de este proyecto es diseñar la estructura de base de datos que sostendrá esta red social, garantizando que sea organizada, rápida y escalable.

Paso a Paso del Diseño del Modelo (Metodología de la Unidad)

Para construir el Modelo entidad relación, seguir los 5 pasos recomendados en el material de estudio:

Paso 1: Identificación de Entidades

Identificamos los objetos y conceptos principales de nuestra red social. Cada entidad representa una "caja" de datos donde agruparemos todo:

- **USUARIO:** Representa a las personas que usan la plataforma (estudiantes y mentores).
- **PUBLICACION:** Mensajes, preguntas, noticias o memes compartidos.
- **GRUPO:** Equipos de estudio, clubes o grupos para hackatones.
- **HABILIDAD:** Temas de conocimiento, deportes o disciplinas.
- **EVENTO:** Reuniones, talleres o actividades organizadas.

Paso 2: Identificación de Relaciones Iniciales

Analizamos cómo interactúan las entidades entre sí:

- Un usuario **crea** publicaciones.
- Un usuario **organiza** eventos.
- Un usuario **pertenece** a grupos.
- Un usuario **tiene** habilidades.
- Un usuario **sigue** a otros usuarios (para asesorías).

Paso 3: Elaboración del Modelo Inicial

Representamos las entidades y sus conexiones mediante líneas para verificar que la información fluya correctamente sin duplicar conceptos.

Paso 4: Identificación de Atributos y Claves

Asignamos las características a cada entidad, aplicando la simbología aprendida:

- **# (Clave Primaria / PK):** El dato único que no se repite (ej. #ID_Usuario).
- *** (Obligatorio):** Datos que el usuario debe ingresar obligatoriamente (ej. *Correo).
- **.** (Opcional): Datos que pueden quedar en blanco (ej. .Descripcion).
- **FK (Clave Foránea):** Copia de un ID de otra tabla para crear la conexión.

Paso 5: Romper relaciones N:M

Se dice que las bases de datos no funcionan bien si conectamos "Muchos a Muchos" directamente. Para arreglar esto, cree **entidades puente para manejar esto**:

MIEMBRO_GRUPO: Conecta a USUARIO con GRUPO. Permite que un estudiante esté en varios grupos y que un grupo tenga varios estudiantes.

PERFIL_HABILIDAD: Conecta a USUARIO con HABILIDAD. Registra las habilidades de cada estudiante y su nivel de dominio.

SEGUIDOR: Conecta a USUARIO con USUARIO (relación recursiva). Permite el sistema de conexiones y mentorías entre estudiantes.

3. Justificación del diseño usado

- **Uso de la entidad puente SEGUIDOR:** En lugar de crear una entidad separada para "Mentores", decidimos manejar una sola entidad USUARIO con el atributo *Rol y conectarla consigo misma mediante SEGUIDOR. Esto simplifica la base de datos y permite que cualquier usuario pueda ser estudiante en una materia y mentor en otra.
- **Flexibilidad en Publicaciones y Eventos:** Añadimos el atributo opcional .Descripcion en PUBLICACION y EVENTO. Esto evita la rigidez y permite que los estudiantes publiquen textos cortos o descripciones largas según la necesidad.
- **Normalización y Escalabilidad:** Al transformar todas las relaciones a 1:N, evitamos que los datos se repitan. Esto garantiza que cuando el sistema crezca o se programe el código backend (por ejemplo, en Python), las búsquedas y consultas sean muy rápidas y no consuman recursos de más.

Conclusiones personales

Construir este diseño fue útil para comprender que el desarrollo de software no comienza escribiendo código, sino entendiendo la arquitectura de los datos. Si la base de datos está mal diseñada desde el inicio, el código que se construya sobre ella será frágil y propenso a errores, luego el mantenimiento del mismo será aún más tedioso.

Si voy a ser un desarrollador de software, este proyecto me enseñó la importancia de la abstracción: saber transformar necesidades del mundo real (como buscar un tutor para una materia) en estructuras de datos organizadas. Modelar este diagrama mejoró lo que es mi capacidad para analizar problemas complejos y proponer soluciones técnicas eficientes, una habilidad clave para crear sistemas robustos y profesionalmente competitivos en la industria actual.